

Approximating $r_{\max}$ from the Problem Instance and the BBLM Channel

This note works out, in detail, how to approximate the repair budget $r_{\max}$ introduced in *Structure of the Prediction Error* (and used throughout *Structured Syndrome Decoding from Active Rows*) directly from the problem instance (n, k, q) and the BBLM bit-flip parameters (α, β) , instead of treating $r_{\max}$ as a value chosen by hand.

It ends with a fully worked numerical example for

$n = 128, k = 64, q = 127, \alpha = 0.01, \beta = 0.05$

giving a concrete estimate $r_{\max} \approx 17$ (at 99% confidence) for that instance, together with the reasoning and the code used to get there.

1. What we are trying to estimate

Recall from the paper: for a codeword $v \in C$ with active support $A = \text{Supp}(v)$, the monomial approximation $\widehat{Q}$ is wrong on some subset of the active rows,

$$E(v) = \{i \in A : \widehat{Q}_i \neq Q_i\}, \quad r(v) = |E(v)|.$$

The repair module is configured to tolerate at most $r_{\max}$ such errors: the structured decoder enumerates hypotheses of dimension up to $r_{\max}$ ($\mathcal{H}_{\leq r_{\max}}$), and the induced syndrome decoder uses radius $\tau = 2r_{\max}$. Right now $r_{\max}$ is just an input parameter. The goal of this note is to turn it into a **derived quantity**: given the instance and the leakage quality, how many active-row errors should we actually *expect*, and how large a budget do we need to cover that with high probability?

The derivation has three layers:

1. **Per-entry channel.** How reliable is a single leaked byte of the monomial matrix?
 2. **Per-row error probability.** Given that, how often does the monomial-approximation construction (**RowScore** / the argmax over candidate rows) get an *entire row* wrong?
 3. **Per-codeword aggregation.** Given that, how many wrong rows should we expect among the $t = \text{wt}(v)$ active rows of a typical low-weight codeword — and what budget $r_{\max}$ covers the bad case with high probability, without making the repair search too expensive?
-

2. The BBLM channel model (concrete instantiation)

This section pins down, concretely, the general "Bit-Based Leakage Model" already described abstractly in *A General Posterior Leakage Model*: an injective encoding $\text{enc} : \mathcal{X} \rightarrow \{0, 1\}^r$, with the leakage providing independent noisy bits of that encoding.

For this paper's object (the monomial matrix Q), the concrete picture is:

- Every entry $Q_{i,j} \in \mathbb{F}_q$ is stored as a fixed-width bit string $\text{enc}(Q_{i,j}) \in \{0, 1\}^b$ — the standard binary representation of the integer representative, padded with leading zeros to b bits. **b is a property of your**

implementation, not of this analysis — the default, minimal choice is $b_{\min} = \lceil \log_2 q \rceil$ (e.g. $b_{\min} = 7$ for $q = 127$, $b_{\min} = 3$ for $q = 7$), but an implementation is free to pad wider than that (e.g. store $q = 127$ in a full byte, $b = 8$). Everything below is written generically in b .

- $\text{enc}(0) = \underbrace{0 \cdots 0}_b$.
- The leakage observes **every entry of the matrix**, independently, through a bit-flip channel: each bit of $\text{enc}(Q_{i,j})$ is flipped $0 \rightarrow 1$ with probability α and $1 \rightarrow 0$ with probability β , independently of every other bit (of that entry and of every other entry).

This matches the paper's own entrywise posterior $p_{i,j}(a) = \Pr[Q_{i,j} = a \mid \mathcal{L}]$ exactly: each entry gets its own leaked byte/bit-string, so each entry gets its own posterior over $\mathbb{F}_q$, consistent with the coordinate-wise independence already assumed in *A General Posterior Leakage Model*.

With a uniform prior over field values, Bayes' rule gives, for the leaked bit-string y at position (i, j) ,

$$p_{i,j}(a) \propto \Pr[y \mid \text{enc}(a)].$$

(Aside: whenever $2^b > q$ — which happens for $b = b_{\min}$ unless q is itself a power of two, and happens more so for any wider padding — a leaked string can, by noise, land on an invalid codepoint. This is exactly the "encoding not onto the full cube" case flagged in the general leakage model, handled there by renormalizing over valid values. That renormalization constant is shared by every candidate at the same position, so it cancels out of every comparison used below — it never actually needs to be computed.)

Does the padding width b actually matter? Only sometimes. If padding is with leading zeros (the natural, standard choice), every *valid* candidate value has the same padding bits (all 0), so those extra bits never differ between any two candidates — and Step 1 below shows that only *differing* bits ever carry discriminating information. So padding beyond $b_{\min}$ with leading zeros never changes p_{err} : computing everything with $b = b_{\min}$ gives the exact same answer as computing it with a wider zero-padded b . (This is confirmed numerically for $q = 127$ in Section 8.2: $b = 7$ and $b = 8$ give identical results to six decimal places.) The one thing that *does* change p_{err} is $b_{\min}$ itself, since that's set by q — a smaller field genuinely has fewer bits' worth, and fewer candidates' worth, of confusion to worry about. Use $b = b_{\min} = \lceil \log_2 q \rceil$ throughout unless you know your implementation pads with something other than leading zeros.

3. Step 1: the row-hypothesis comparison collapses to two entries

Fix an active row i with true hypothesis (j^*, a^*) (column j^* holds the nonzero value a^*). The **RowScore** algorithm scores every candidate (j, a) by

$$\gamma_i(j, a) = \log p_{i,j}(a) + \sum_{\ell \neq j} \log p_{i,\ell}(0).$$

Because a monomial row agrees with any other candidate row on **all but two** columns (both say "zero" everywhere except their one nonzero position), the comparison telescopes: define

$$\Delta_\ell(c) := \log p_{i,\ell}(c) - \log p_{i,\ell}(0),$$

("how much does column ℓ 's leaked byte support value c over zero"). A short calculation (expand both scores, cancel the $n - 2$ shared zero terms) gives, for **any** candidate $(j, a) \neq (j^*, a^*)$:

$$\gamma_i(j, a) - \gamma_i(j^*, a^*) = \Delta_j(a) - \Delta_{j^*}(a^*).$$

So a wrong candidate wins exactly when its Δ -score beats the true entry's Δ -score. This holds uniformly whether $j = j^*$ (competing on *which value* the true column holds) or $j \neq j^*$ (competing on *which column* is

nonzero). It reduces an n -column comparison to a two-entry comparison, and it is the key simplification that makes the rest of this tractable.

4. Step 2: the channel's discriminating power — $\rho(\alpha, \beta)$

4.0 A primer: what is the Bhattacharyya bound?

Before using it, here's what problem it solves and where it comes from — this is standard machinery from detection theory, not something specific to this paper, so it's worth building intuition for on its own.

The setup. Suppose you need to decide between two competing explanations for something — statisticians call these *hypotheses*, H_0 and H_1 . You don't see the truth directly; you only see a noisy observation y , and you know how likely each possible y is under each hypothesis: $\Pr[y \mid H_0]$ and $\Pr[y \mid H_1]$. The best possible strategy (the *maximum-likelihood*, or ML, decision rule) is simple: pick whichever hypothesis makes the y you actually saw more likely. Even with this best possible strategy, you will sometimes be wrong — noise can occasionally produce an observation that "looks like" the wrong hypothesis.

The question the bound answers. How often does even the *best* decision rule get it wrong? Computing this exactly can be fiddly (it depends on precisely where the "decide H_0 " vs. "decide H_1 " boundary falls, which can be an awkward calculation). The Bhattacharyya bound sidesteps that by giving a simple, always-valid upper bound on that error probability, built directly from the two distributions:

$$\Pr[\text{ML rule picks the wrong hypothesis}] \leq \sum_y \sqrt{\Pr[y \mid H_0] \Pr[y \mid H_1]}.$$

Why this particular formula. The quantity on the right — called the *Bhattacharyya coefficient* — measures how much the two distributions overlap.

- If H_0 and H_1 produce very different-looking observations (whenever $\Pr[y \mid H_0]$ is large, $\Pr[y \mid H_1]$ is tiny, and vice versa), every product $\Pr[y \mid H_0] \Pr[y \mid H_1]$ is small, so the sum is small — the two hypotheses are easy to tell apart, and the error bound is small. Good.
- If H_0 and H_1 produce nearly identical observations ($\Pr[y \mid H_0] \approx \Pr[y \mid H_1]$ for every y), the sum approaches $\sum_y \Pr[y \mid H_0] = 1$ — the two hypotheses are nearly indistinguishable, and the bound is close to its worst possible value (no better than guessing). Bad, but correctly so.

So the coefficient is a single number capturing "how confusable are these two hypotheses," computed purely from their two output distributions — no need to actually simulate or run the decision rule to get a handle on how well it will do.

A tiny numeric example, using your own channel numbers. Say we're trying to tell "the true bit is 0" apart from "the true bit is 1," given one noisy read y through your channel ($\alpha = 0.01$ flips $0 \rightarrow 1$, $\beta = 0.05$ flips $1 \rightarrow 0$):

	$y = 0$	$y = 1$
$\Pr[y \mid \text{true} = 0]$	0.99	0.01
$\Pr[y \mid \text{true} = 1]$	0.05	0.95

The Bhattacharyya coefficient is $\sqrt{0.99 \times 0.05} + \sqrt{0.01 \times 0.95} \approx 0.222 + 0.097 \approx 0.320$ — a fairly small number, telling us these two hypotheses (bit is 0 vs. bit is

1. are reasonably easy to distinguish from a single noisy bit: the ML rule's error probability on this one bit is at most about 32%. This is exactly the quantity $\rho(\alpha, \beta)$ derived symbolically just below — same

calculation, done with variables instead of your specific numbers.

4.1 The general bound

For a single bit where two hypotheses disagree (one says the bit is 0, the other says 1), the classical **Bhattacharyya bound** bounds the probability that a maximum-likelihood decision favors the wrong hypothesis, from the noisy read of that one bit:

$$\Pr[\text{wrong hypothesis wins this bit}] \leq \sum_{y \in \{0,1\}} \sqrt{\Pr[y | 0] \Pr[y | 1]}.$$

Plugging in the channel ($\Pr[y = 1 | 0] = \alpha$, $\Pr[y = 0 | 1] = \beta$):

$$\rho(\alpha, \beta) := \sqrt{\alpha(1 - \beta)} + \sqrt{\beta(1 - \alpha)}.$$

This single number, between 0 and 1, summarizes "how good is this channel" for the purpose of distinguishing hypotheses. As a sanity check: if the channel were symmetric ($\alpha = \beta = p$), this collapses to $\rho = 2\sqrt{p(1 - p)}$ — the standard Bhattacharyya parameter of a BSC(p) from the channel-coding literature. Since the bit channel is memoryless (independent bits, matching the leakage model's own independence assumption), bits where two candidates *agree* contribute a factor of 1 (no discriminating information), and bits where they *disagree* each contribute a factor of $\rho(\alpha, \beta)$. So for two candidates whose b -bit encodings differ in δ positions (Hamming distance δ),

$$\Pr[\text{wrong candidate wins}] \leq \rho(\alpha, \beta)^\delta.$$

5. Step 3: assembling the per-row error bound

There are two structurally different kinds of competitors for row i :

Same-column competitors ($j = j^*$, a different value $a \neq a^*$ at the true column). Both $\Delta_{j^*}(a)$ and $\Delta_{j^*}(a^*)$ come from the *same* leaked byte, so the direct Bhattacharyya bound applies, using the Hamming distance **between the two candidate encodings directly**:

$$\Pr[\Delta_{j^*}(a) \geq \Delta_{j^*}(a^*)] \leq \rho(\alpha, \beta)^{\delta(a, a^*)}, \quad \delta(a, a^*) := \text{Hamming}(\text{enc}(a), \text{enc}(a^*)).$$

Cross-column competitors ($j \neq j^*$, some nonzero a proposed at a column that is truly zero). Now $\Delta_j(a)$ and $\Delta_{j^*}(a^*)$ come from *independent* leaked bytes. A Chernoff bound at the symmetric point splits into a product of two Bhattacharyya-type terms, each measured against the encoding of zero:

$$\Pr[\Delta_j(a) \geq \Delta_{j^*}(a^*)] \leq \rho(\alpha, \beta)^{\delta(a)} \cdot \rho(\alpha, \beta)^{\delta(a^*)}, \quad \delta(c) := \text{Hamming}(\text{enc}(c), \text{enc}(0)) = \text{popcount}(c).$$

A union bound over all $n(q - 1) - 1$ wrong candidates ($q - 2$ same-column, $(n - 1)(q - 1)$ cross-column) gives

$$p_{\text{err}}(\text{row } i \mid a^*) \leq \underbrace{\sum_{a \neq a^*} \rho^{\delta(a, a^*)}}_{\text{same column}} + \underbrace{(n - 1) \rho^{\delta(a^*)} \sum_{a \in \mathbb{F}_q^*} \rho^{\delta(a)}}_{\text{other columns}}.$$

5.1 Important caveat: this union bound can be vacuous

This bound is only useful when it is comfortably below 1. With n columns and q field values, there are on the order of $n \cdot q$ competitors, and a union bound over that many terms can easily exceed 1 — at which point it says literally nothing (a probability bound above 1 carries no information; the true error probability could be anywhere in $[0, 1]$). This is a well-known weakness of naive union-Bhattacharyya bounds when the "alphabet size" (here,

roughly $n \cdot q$ is large relative to the channel quality. **This actually happens for the worked example below** — see Section 8.2. When it does, the fix used here is the exact computation of Section 6, which has no sampling noise and no looseness at all, rather than a loose analytical bound.

6. Step 3': computing the per-row error probability exactly

When the analytic union bound is vacuous, the fix isn't a tighter bound — a leaked bit-string only takes 2^b possible values, a small, literally enumerable set. That's enough to compute $p_{\text{err}}(\text{row})$ *exactly*, with no approximation and no sampling noise at all.

6.1 The exact computation

1. Precompute a lookup table $\text{LUT}[y][c] = \log \Pr[y \mid \text{enc}(c)] - \log \Pr[y \mid \text{enc}(0)]$ for every possible leaked string $y \in \{0, \dots, 2^b - 1\}$ and every candidate value $c \in \{0, \dots, q - 1\}$ (depends only on α, β, b ; computed once). This is exactly $\Delta_\ell(c)$ from Section 3, tabulated.
2. **True column.** For the active column, the leaked string y^* has a fully known distribution given the true value a^* (it's a product of b independent per-bit channel probabilities). Since there are only 2^b possible y^* , both the true score $\Delta_{j^*}(a^*)$ and the best same-column competitor's score are exact, known functions of y^* — a 2^b -term sum, no approximation.
3. **Other columns.** The other $n - 1$ columns are i.i.d. (each independently has true value 0). For i.i.d. random variables, the distribution of their maximum has a closed form from order statistics: if $F(x)$ is the exact CDF of *one* column's best candidate score (again just a 2^b -point discrete distribution, cheap to compute exactly), then the max over $n - 1$ independent columns has CDF $F(x)^{n-1}$ — exactly, with no need to simulate all $n - 1$ columns individually.
4. Combine: for each of the 2^b possible y^* , either the same-column competitor already beats the truth (certain error), or it doesn't, in which case the error probability contributed is $1 - F(\Delta_{j^*}(a^*))^{n-1}$ using the order-statistics CDF from step 3. Weight by $\Pr[y^* \mid a^*]$ and sum over the 2^b outcomes, then average over the uniformly random true value $a^* \in \mathbb{F}_q^*$ (same averaging justification as Section 7 below).

This computes the exact same event as simulating **RowScore** (the argmax over $\gamma_i(j, a)$, re-expressed through the Δ -scores from Section 3) — just by enumerating the small outcome space directly instead of sampling it.

6.2 Monte Carlo as a cross-check

Simulating the same decision rule directly (sample a^* , leak every column through the channel, take the argmax, check whether it's correct, repeat many times) gives an independent, much simpler-to-implement estimate of the same quantity, useful mainly as a sanity check on the exact computation above — see Section 8.3, where the two agree to within the Monte Carlo run's own sampling error.

7. Step 4: aggregating across a codeword — from p_{err} to r_{max}

Under the entrywise-independence assumption the paper already makes (*A General Posterior Leakage Model*), the per-row error events across the $t = \text{wt}(v)$ active rows of a codeword are approximately independent, so

$$r(v) \sim \text{Binomial}(t, p_{\text{err}}),$$

where p_{err} is the row error probability averaged over the (uniformly random) true value at each active row — averaging, rather than taking a worst case, is the statistically correct choice here because each of the t rows independently draws its own true value, so the *sum* over rows is governed by the mean error rate, not the worst-case one.

$r_{\max}$ is then chosen as a high quantile of this distribution: the smallest r such that

$$\Pr[\text{Binomial}(t, p_{\text{err}}) \leq r] \geq 1 - \delta_{\text{target}}$$

for a target confidence $1 - \delta_{\text{target}}$ (e.g. 99%). This can be read off the exact binomial CDF (cheap for the t values involved here), or approximated via:

- **Chernoff/KL bound** (rigorous): $\Pr[X \geq r] \leq \exp(-t \cdot D(r/t \| p_{\text{err}}))$ with D the binary KL divergence — solve for the smallest r making the right-hand side $\leq \delta_{\text{target}}$.
- **Normal approximation** (quick, less rigorous for small t or small p_{err}): $r_{\max} \approx t p_{\text{err}} + z_{1-\delta} \sqrt{t p_{\text{err}}(1 - p_{\text{err}})}$.

Trade-off to keep in mind. Larger $r_{\max}$ means higher confidence of repair, but the structured decoder's hypothesis count is $N_{\text{hyp}} = O(t^{r_{\max}} K^{r_{\max}})$ (from the complexity analysis in *Structured Syndrome Decoding from Active Rows*), so $r_{\max}$ should be picked as small as the desired success probability allows, not maximized blindly. Choosing the confidence level $1 - \delta_{\text{target}}$ is therefore itself a design knob, not something the theory pins down uniquely.

7.1 Where does t come from?

The target weight t of enumerated codewords is a design choice of the enumeration module (θ_{enum} in the paper), left abstract in the current draft. A standard, defensible default is to target codewords near the code's minimum distance, estimated via the (non-asymptotic) **Gilbert–Varshamov bound**: the largest d such that

$$\sum_{i=0}^{d-2} \binom{n-1}{i} (q-1)^i < q^{n-k}.$$

This is the choice used in the worked example below; if your enumeration module actually targets a different weight, substitute it directly — every other quantity in this note (p_{err}, ρ) is independent of t .

8. Worked example

Instance: $n = 128, k = 64, q = 127, \alpha = 0.01, \beta = 0.05$. Minimal bit-width $b_{\min} = \lceil \log_2 127 \rceil = 7$; the implementation described actually pads to a full byte, $b = 8$ — both are computed below to confirm they agree, per the "does padding matter" discussion in Section 2.

8.1 Channel quality

$$\rho(\alpha, \beta) = \sqrt{0.01 \times 0.95} + \sqrt{0.05 \times 0.99} = 0.319954.$$

(This number doesn't depend on b — it's purely a property of the channel, one bit at a time.)

8.2 Analytic union bound — and why it is vacuous here

Computing $S = \sum_{a \in \mathbb{F}_q^*} \rho^{\delta(a)} \approx 5.98$ and averaging the bound of Section 5 over a uniformly random a^* gives

term	value
$\mathbb{E}_{a^*}[\text{cross-column term}]$	≈ 36.05
$\mathbb{E}_{a^*}[\text{same-column term}]$	≈ 5.89
analytic bound on p_{err}	≈ 41.9

This is far above 1 — with $n = 128$ columns and $q = 127$ candidate values per column (over 16,000 total competitors), the union bound over that many Bhattacharyya terms is simply too loose at this channel quality to say anything useful. This is *not* evidence that the true error rate is high; it just means we need the exact computation of Section 6 instead.

8.3 Exact computation (and a Monte Carlo cross-check)

Running the order-statistics computation from Section 6.1:

bit-width	unused codepoints	p_{err} (exact)
$b = 7$ (minimal, $b_{\min}$)	1 of 128	0.203656
$b = 8$ (byte, as implemented)	129 of 256	0.203656

Identical to six decimal places, exactly as the padding argument in Section 2 predicts — the 8th bit is 0 for every valid candidate, so it never enters any Δ -comparison, noise on it or not. **So it is always safe to use $b = b_{\min}$ for this analysis, regardless of how wide your implementation's actual storage is**, as long as the padding is with leading zeros.

A 200,000-trial Monte Carlo simulation of the same decision rule (Section 6.2), run as an independent cross-check, gives $p_{\text{err}} \approx 0.2046 \pm 0.0009$ — matching the exact value to within its own sampling error, as it should.

So, at this leakage quality, the monomial-approximation construction gets roughly **1 in 5 active rows wrong** — noisy, but far from useless (compare to a uniform random guess among 16,129 candidates, which would be wrong essentially always).

8.4 Target weight

The Gilbert–Varshamov bound for $(n, k, q) = (128, 64, 127)$ gives

$$t = d_{\text{GV}} = 49.$$

8.5 Aggregating to r_{max}

With $r(v) \sim \text{Binomial}(t=49, p_{\text{err}}=0.203656)$: mean ≈ 9.98 , standard deviation ≈ 2.82 .

target confidence	exact binomial quantile r_{max}	normal approximation
90%	14	13.59
95%	15	14.62
99%	17	16.54
99.9%	19	18.69

8.6 Headline answer

$r_{\text{max}} \approx 17$
(at 99% confidence, for $n=128, k=64, q=127, \alpha=0.01, \beta=0.05$)

i.e. configure the repair module to tolerate up to 17 active-row errors, and the structured decoder will cover the true error dimension for at least 99% of enumerated weight-49 codewords under this leakage quality. This is the same headline number as before the exact-computation refinement — the tiny shift in p_{err} ($0.2046 \rightarrow 0.2037$) wasn't enough to move the discrete quantile — but it's now backed by an exact calculation rather than a

simulation. If you use a different confidence target or a different rule for t , read the corresponding value off the table in 8.5, or rerun the code in the Appendix with the new numbers.

8.7 A smaller field, for contrast: $q = 7$

Since the field size sets $b_{\min} = \lceil \log_2 q \rceil$, a smaller field genuinely changes the answer — unlike padding, which doesn't. For illustration, take $n = 32, k = 16, q = 7$ (so $b_{\min} = 3$) at the *same* channel quality $\alpha = 0.01, \beta = 0.05$:

quantity	value
$b_{\min} = \lceil \log_2 7 \rceil$	3
t (Gilbert–Varshamov)	10
p_{err} (exact)	0.0875
r_{max} at 99% confidence	3

The per-row error rate is much lower than in the $q = 127$ example (0.09 vs. 0.20) at the *same* α, β — not because the channel is better, but because there are far fewer competing (j, a) hypotheses to confuse the truth with ($n(q - 1) = 192$ here, vs. 16,129 for $q = 127$). Smaller alphabets are intrinsically easier to decode correctly, all else equal.

9. Assumptions used throughout (so they're easy to challenge)

- Uniform prior over field values for each entry (used to justify that the Bayesian normalization constant cancels inside every Δ -comparison).
- Standard binary representation for `enc`, with `enc(0) = 0...0`.
- Bit-width $b = \lceil \log_2 q \rceil$ (the minimal representation) — proven safe to use as a stand-in for any wider, zero-padded implementation choice (Section 2), and confirmed numerically for $q = 127$ in Section 8.3.
- Independence of leakage across different bits, different entries, and (for the aggregation step) different rows — all inherited directly from assumptions already made elsewhere in the paper.
- t via the Gilbert–Varshamov bound, standing in for whatever weight rule the enumeration module actually uses.
- The 99% confidence target in the headline number is a choice, not a derived constant — trade it against the N_{hyp} decoding cost as needed.

Appendix: reproducing the numbers

```
import math
import numpy as np
from math import comb
from scipy.stats import binom

def rho(alpha, beta):
    return math.sqrt(alpha*(1-beta)) + math.sqrt(beta*(1-alpha))

def gv_weight(n, k, q):
    """Non-asymptotic Gilbert–Varshamov target weight t."""
    target = q**(n-k)
    def vol(m, r):
```

```

        return sum(comb(m, i) * (q-1)**i for i in range(r+1))
d = 1
while True:
    r = d - 2
    v = vol(n-1, r) if r >= 0 else 0
    if v < target:
        d += 1
    else:
        d -= 1
        break
return d

def p_err_exact(q, alpha, beta, n, b=None):
    """
    Exact per-row error probability (Section 6.1), generic in the bit-width
    b. Defaults to the minimal width b_min = ceil(log2(q)); per Section 2,
    any wider zero-padded b gives an identical result, so the default is
    always the right (and cheapest) choice unless your encoding pads with
    something other than leading zeros.
    """
    if b is None:
        b = math.ceil(math.log2(q))
    B = 2**b
    bits = np.array([(c >> i) & 1 for i in range(b)] for c in range(B))

    term_y1 = np.log((1 - beta) / alpha) # candidate bit = 1, observed y_bit =
1
    term_y0 = np.log(beta / (1 - alpha)) # candidate bit = 1, observed y_bit =
0

    LUT = np.zeros((B, q)) # LUT[y][c] = Delta(y,c)
    c_bits = bits[:q]
    for y in range(B):
        contrib = np.where(bits[y] == 1, term_y1, term_y0)
        LUT[y] = (c_bits * contrib).sum(axis=1)

    def leak_dist(c):
        cb = bits[c]
        p1 = np.where(cb == 1, 1 - beta, alpha) # P(y_bit=1 | true_bit)
        p0 = 1 - p1
        probs = np.zeros(B)
        for y in range(B):
            yb = bits[y]
            probs[y] = np.where(yb == 1, p1, p0).prod()
        return probs

    dist0 = leak_dist(0)
    best_single = LUT.max(axis=1) # best candidate score at one
    "true=0" column
    order = np.argsort(best_single)
    sorted_vals = best_single[order]
    cum_probs = np.cumsum(dist0[order]) # exact CDF F_single, tabulated

    def F_single(x):
        idx = np.searchsorted(sorted_vals, x, side='right') - 1
        return 0.0 if idx < 0 else cum_probs[idx]

    n_other = n - 1

```

```

def p_err_given_astar(astar):
    dist_astar = leak_dist(astar)
    score_true = LUT[:, astar]
    same = LUT.copy(); same[:, astar] = -np.inf
    best_same = same.max(axis=1)
    total = 0.0
    for y in range(B):
        py = dist_astar[y]
        if py == 0:
            continue
        s = score_true[y]
        if best_same[y] > s:
            total += py # same-column beats
    truth: certain error
    else:
        total += py * (1 - F_single(s) ** n_other) # else: chance some
other column beats it
    return total

return np.mean([p_err_given_astar(a) for a in range(1, q)])

def r_max_table(t, p_err, confs=(0.90, 0.95, 0.99, 0.999)):
    return {c: int(binom.ppf(c, t, p_err)) for c in confs}

# --- worked example: n=128, k=64, q=127 ---
n, k, q, alpha, beta = 128, 64, 127, 0.01, 0.05
t = gv_weight(n, k, q)
p_err = p_err_exact(q, alpha, beta, n) # uses b_min = 7 automatically
p_err_byte = p_err_exact(q, alpha, beta, n, b=8) # explicit byte width, for
comparison
print("rho =", rho(alpha, beta))
print("t (GV) =", t)
print("p_err (b=7, minimal) =", p_err)
print("p_err (b=8, byte) =", p_err_byte) # identical to 6 decimals
print("r_max table:", r_max_table(t, p_err))

# --- smaller-field example: n=32, k=16, q=7 (Section 8.7) ---
n2, k2, q2 = 32, 16, 7
t2 = gv_weight(n2, k2, q2)
p_err2 = p_err_exact(q2, alpha, beta, n2) # uses b_min = 3 automatically
print("\nq=7 example: t =", t2, " p_err =", p_err2, " r_max table:",
r_max_table(t2, p_err2))

```